



J. J. Strossmayer University of Osijek

**Faculty of Electrical Engineering,
Computing and Information
Technologies Osijek**
Kneza Trpimira 2b
HR-31000 Osijek
www.ferit.unios.hr

Lab Assignment 1: **Client-Server Architecture**

Academic year 2025/2026.

The Day the Internet Went Silent (For Half an Hour)

*In 1988, a graduate student named Robert Tappan Morris created a program intended to measure the size of the early internet. This program, later known as the **Morris Worm**, was a simple client-server application: it sent requests to machines to test if they were already running a copy of the worm. If not, the server on the machine would allow the worm to propagate.*

Unfortunately, a small bug in the code caused the worm to spread uncontrollably, infecting multiple machines and overloading their resources. Within hours, 10% of the world's internet-connected computers (around 6,000 machines at the time) became sluggish or unusable.

*The incident led to the creation of the **Computer Emergency Response Team (CERT)** and highlighted the importance of understanding client-server behavior, testing software thoroughly, and anticipating unintended consequences.*

Exercise 1: Client-Server Architecture

1 Introduction to Client-Server Architecture

1.1 The roles of client and server.

In the client-server architecture, two distinct roles form the foundation of communication in distributed systems: the client and the server. Each plays a unique and complementary role, ensuring the seamless exchange of data and functionality.

1.1.1 Client

The client is an entity or process that initiates communication by sending requests to the server. Its primary purpose is to consume services or resources provided by the server. Clients are typically user-facing applications or devices that rely on the server to handle tasks requiring centralized processing or data storage.

Key characteristics of clients include:

- **Request Initiation:** The client actively starts the communication by sending a request to the server.
- **Resource Consumption:** Clients depend on servers for resources such as data, computations, or services.
- **Asynchronous Interaction:** Clients may continue other operations while awaiting a response from the server.

Examples:

- A web browser (client) requests web pages from a web server.
- A mobile app fetches data from a remote database server via an API.

1.1.2 Server

The server is an entity or process designed to provide services or resources in response to client requests. Servers are typically optimized for performance and reliability to handle multiple clients simultaneously.

Key characteristics of servers include:

- **Service Provision:** The server responds to requests from clients, often by retrieving or processing data.
- **Passive Behavior:** The server waits for incoming requests rather than initiating communication.
- **Concurrent Handling:** Servers are designed to manage multiple requests concurrently, ensuring scalability and reliability.

Examples:

- A web server hosts websites and responds to HTTP requests.
- A database server processes queries and returns requested data.

1.2 Importance of the Client-Server Model in Distributed Systems

The client-server model is essential to the distributed systems, providing a structured approach to the organization of networked components. Its importance lies in its ability to facilitate scalable, modular, and reliable systems that cater to a wide range of applications, from web, to IoT and cloud.

The client-server model serves as a fundamental design paradigm in distributed systems. At its core, the model organizes system components into distinct roles, enabling efficient collaboration between clients, which request services, and servers, which fulfill those requests. This enables creating scalable, maintainable, and versatile systems.

One of the primary strengths of the client-server model lies in its modularity and clear separation of concerns. By delegating user-facing tasks to clients and resource-intensive operations to servers, the model simplifies system design and allows developers to focus on specific aspects of functionality. This modularity not only facilitates easier implementation but also supports ongoing maintenance, as updates to the server, such as improvements to

data processing logic, can be made independently of the client applications. Scalability is another critical advantage of the client-server model, making it suitable for systems that must accommodate a growing number of users or devices. Centralizing services on the server enables efficient resource utilization and paves the way for scaling strategies such as load balancing and server replication. As demand increases, additional servers can be introduced to handle client requests without requiring significant modifications to the client-side software. The model also excels in optimizing resource usage by centralizing data storage and processing. Instead of duplicating resources across multiple clients, servers act as shared hubs, providing consistent access to data and services. This centralized approach reduces redundancy and ensures uniformity, which is particularly beneficial in applications where data integrity and synchronization are critical.

In addition to these technical benefits, the client-server model supports flexibility and interoperability. By leveraging standardized communication protocols, it allows clients and servers to operate across diverse platforms and environments. This capability is especially valuable in modern distributed systems, where heterogeneous devices—from mobile phones to IoT sensors—must interact seamlessly with centralized servers. Furthermore, centralizing services within a server simplifies system management and control. Tasks such as security updates, backups, and configuration changes can be performed at the server level, ensuring consistency and reducing the risk of errors. This centralized control enhances reliability and makes the client-server model an appealing choice for applications where stability is paramount. As a foundational element of distributed computing, the client-server model underpins a wide array of technologies, including web applications, email systems, and cloud-based services. It also provides the basis for more advanced architectures, such as multi-tier systems and microservices, which extend the principles of modularity and scalability. By enabling efficient communication and resource sharing, the client-server model has become an indispensable framework for building robust and adaptable distributed systems.

Through its combination of simplicity, flexibility, and efficiency, the client-server model continues to be a cornerstone of distributed systems. Its enduring relevance is a testament to its ability to meet the demands of modern applications while remaining adaptable to the evolving landscape of networked computing.

2 Key Concepts

2.1 Sockets and their role in communication.

2.1.1 Understanding Sockets

A socket serves as the foundation for communication in distributed systems, acting as an endpoint for sending and receiving data between devices over a network. It is an abstraction provided by the operating system, allowing applications to establish connections and exchange information without needing to handle the complexities of the underlying network hardware.

Sockets are a critical part of the client-server model. On the server side, a socket listens for incoming client requests, while on the client side, a socket is used to initiate communication with the server. Together, these sockets enable bidirectional data exchange over protocols such as TCP (Transmission Control Protocol) or UDP (User Datagram Protocol).

A socket is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to. An endpoint is a combination of an IP address and a port number.

2.1.2 Types of Sockets

Sockets can be classified into two primary types based on the protocol used for communication:

1. Stream Sockets (TCP)

Characteristics: Reliable, connection-oriented, and ensures data is delivered in order without loss.

Use Case: Applications requiring consistent data delivery, such as file transfers, web browsers, and email systems.

Example: A web browser opening a TCP connection to a web server.

A stream socket provides bidirectional, reliable, sequenced, and unduplicated flow of data with no record boundaries. After the connection has been established, data can be read from and written to these sockets as a byte stream. The socket type is `SOCK_STREAM`.

Functions in Stream Socket Communication

1. Socket Creation

```
int sock_fd = socket(AF_INET, SOCK_STREAM, 0);
```

2. Connection Setup

Client: Connect to a server using `connect()`.

```
connect(sock_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
```

Server: Bind to an address and port, then listen for incoming connections with `listen()` and accept them with `accept()`.

```
bind(sock_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));  
listen(sock_fd, backlog);  
int client_fd = accept(sock_fd, (struct sockaddr*)&client_addr, &addr_len);
```

3. Sending and Receiving Data

Send data using `send()`.

```
send(sock_fd, data, data_length, 0);
```

Receive data using `recv()`.

```
recv(sock_fd, buffer, buffer_size, 0);
```

4. Connection Termination

```
close(sock_fd);
```

2. Datagram Sockets (UDP)

Characteristics: Unreliable, connectionless, and focuses on low-latency communication. Delivery and order are not guaranteed.

Use Case: Applications prioritizing speed over reliability, such as online gaming, video streaming, or DNS lookups.

Example: A video conferencing app transmitting voice packets via UDP.

Datagram sockets provide a mechanism for sending and receiving packets of data without establishing a connection between the sender and receiver. They are used in connectionless communication, where data is transmitted in discrete units called datagrams. These sockets operate over the User Datagram Protocol (UDP), which is part of the Internet Protocol suite. The socket type is `SOCK_DGRAM`.

Functions in Datagram Socket Communication

1. Socket Creation

```
int sock_fd = socket(AF_INET, SOCK_DGRAM, 0);
```

2. Sending Data

```
sendto(sock_fd, data, data_length, 0, (struct sockaddr*)&dest_addr, sizeof(dest_addr));
```

3. Receiving Data

```
recvfrom(sock_fd, buffer, buffer_size, 0, (struct sockaddr*)&src_addr, &addr_len);
```

4. Binding

```
bind(sock_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
```

2.1.3 The Role of Sockets in Client-Server Communication

Sockets play a central role in facilitating communication between clients and servers. The typical workflow includes the following steps:

1. Server Setup:
 - A server creates a socket and binds it to a specific address (IP and port).
 - It then listens for incoming connections from clients.
2. Client Connection:
 - The client creates its socket and attempts to connect to the server using the server's address.
 - If the connection is successful, the server accepts the client's request.
3. Data Exchange:
 - Once connected, the client and server use their respective sockets to send and receive data.
 - This exchange continues until one party closes the connection.
4. Termination:
 - After the communication ends, the sockets are closed to free up system resources.
 - Protocols (e.g., TCP/IP and UDP).
 - Request-response model vs. persistent connections.

2.1.4 Example: Echo Server

A classic demonstration of socket communication is an Echo Server, where the server simply sends back whatever message it receives from the client.

- Server Side (Pseudocode):

```
socket = create_socket();
bind(socket, server_address, port);
listen(socket);
client_socket = accept_connection(socket);
while (data = receive(client_socket)) {
    send(client_socket, data);
}
close(socket);
```

- Client Side (Pseudocode):

```
socket = create_socket();
connect(socket, server_address, port);
send(socket, "Hello, Server!");
response = receive(socket);
close(socket);
```

3 Basic Server Program in C++

The basic server program code for the Echo Server.

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in address = {AF_INET, htons(PORT), INADDR_ANY};
    bind(server_fd, (sockaddr*)&address, sizeof(address));
    listen(server_fd, 3);

    std::cout << "Server is running on port " << PORT << " and waiting for connections...\n";

    int client_fd = accept(server_fd, nullptr, nullptr);
    if (client_fd >= 0) {
        std::cout << "Client connected.\n";
    }

    char buffer[BUFFER_SIZE];
    while (true) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytes_received = read(client_fd, buffer, BUFFER_SIZE - 1);

        if (bytes_received <= 0) {
            std::cout << "Client disconnected.\n";
            break;
        }

        std::cout << "Received from client: " << buffer << " --> ";

        send(client_fd, buffer, bytes_received, 0);
        std::cout << "Echoed back to client: " << buffer << "\n";
    }

    close(client_fd);
    close(server_fd);
    std::cout << "Server shut down.\n";

    return 0;
}
```

The basic server program code for the Echo Server that supports multiple clients.

```
#include <iostream>
#include <cstring>
#include <thread>
#include <vector>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define PORT 8080
#define BUFFER_SIZE 1024

void handle_client(int client_fd) {
    char buffer[BUFFER_SIZE];
    while (true) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytes_received = read(client_fd, buffer, BUFFER_SIZE - 1);

        if (bytes_received <= 0) {
            std::cout << "Client disconnected.\n";
            break;
        }

        std::cout << "Received from client: " << buffer << " --> ";
        send(client_fd, buffer, bytes_received, 0);
        std::cout << "Echoed back to client: " << buffer << "\n";
    }

    close(client_fd);
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in address = {AF_INET, htons(PORT), INADDR_ANY};
    int option_value = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &option_value, sizeof(int));
    bind(server_fd, (sockaddr*)&address, sizeof(address));
    listen(server_fd, 3);

    std::cout << "Server is running on port " << PORT << " and waiting for connections...\n";

    std::vector<std::thread> client_threads;

    while (true) {
        int client_fd = accept(server_fd, nullptr, nullptr);
        if (client_fd >= 0) {
            std::cout << "Client connected.\n";
            // Start a new thread to handle the client
            client_threads.emplace_back(handle_client, client_fd);
        }
    }

    // Wait for all client threads to finish (never reached in this infinite loop)
    for (auto& t : client_threads) {
        t.join();
    }

    close(server_fd);
    return 0;
}
```


4 Basic Client Program in C++

The client that connects to the Echo Server (if the server supports multiple client connections this code can be run multiple times and/or on multiple machines):

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int client_fd = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in server_address = {};
    server_address.sin_family = AF_INET;
    server_address.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_address.sin_addr);

    if (connect(client_fd, (sockaddr*)&server_address, sizeof(server_address)) == 0) {
        std::cout << "Connected to the server at 127.0.0.1:" << PORT << "\n";
    } else {
        std::cerr << "Failed to connect to the server.\n";
        return 1;
    }

    char buffer[BUFFER_SIZE];
    while (true) {
        std::cout << "Enter a message to send to the server (type 'exit' to quit): ";
        std::string message;
        std::getline(std::cin, message);

        if (message == "exit") {
            std::cout << "Closing connection...\n";
            break;
        }

        // Send the message to the server
        send(client_fd, message.c_str(), message.size(), 0);
        std::cout << "Sent to server: " << message << "\n";

        // Receive the echo response from the server
        memset(buffer, 0, BUFFER_SIZE);
        int bytes_received = read(client_fd, buffer, BUFFER_SIZE - 1);

        if (bytes_received > 0) {
            std::cout << "Received from server: " << buffer << "\n";
        } else {
            std::cout << "Server disconnected.\n";
            break;
        }
    }

    close(client_fd);
    std::cout << "Client shut down.\n";

    return 0;
}
```

5 Assignments

Assignment 1. Converting a Multi-Client Server to a Publish-Subscribe System

Transform the provided multi-client server from and client code from the example into a fully functional Publish-Subscribe (Pub-Sub) System, where clients can:

- Subscribe to specific topics of interest.
- Publish messages to topics they choose.
- Automatically receive messages for topics they are subscribed to.

On the server side, implement topic management to keep track of which clients are subscribed to which topics. This involves handling SUBSCRIBE commands to add clients to specific topics and PUBLISH commands to distribute messages to the appropriate subscribers. The server must relay messages published to a topic to all clients subscribed to that topic.

You can maintain a mapping of topics to their subscribers using a suitable data structure (e.g., `std::map<std::string, std::vector<int>>>`)

On the client side, add functionality to allow clients to subscribe to topics by sending the appropriate commands to the server. Additionally, implement the ability for clients to publish messages to specific topics. The client should also be able to display any messages it receives for topics it is subscribed to, ensuring a seamless interaction between the client and the server.

Implement handling for the following commands sent by clients:

- SUBSCRIBE <topic>: Adds the client to the subscriber list for the specified topic.
- PUBLISH <topic> <message>: Forwards the message to all clients subscribed to the topic.

Relay messages to all clients in the subscriber list for a topic.

Example:

- SUBSCRIBE news: Subscribes the client to the "news" topic.
- PUBLISH news Breaking news: Croatia Tops EU Inflation Rankings for 2025: Publishes a message to the "news" topic.

Testing

Test the system with multiple clients connected to the server. Demonstrate the following scenarios:

- A client subscribes to multiple topics and receives messages published to those topics.
- Multiple clients subscribe to the same topic and receive all published messages.
- A client publishes messages to topics that have no subscribers.

Assignment 2. Implementing advanced features to the Publish-Subscribe System

Thread Safety:

If multiple threads modify the topic-subscriber mapping, use a mutex to ensure safe access.

Unsubscribe Functionality

Extend the system to allow clients to unsubscribe from topics. This involves implementing a new command, UNSUBSCRIBE <topic>, which removes the client from the subscriber list for the specified topic. Update the server to handle this command and ensure that messages published to the topic are no longer sent to unsubscribed clients.

Assignment 3. BONUS Assignment: Topic Listing

Add a command, LIST TOPICS, that allows clients to request a list of all active topics on the server. The server should return the names of all topics currently being managed, regardless of whether they have subscribers.